Mitigating False Credit Propagation: Probabilistic Graphical Reward Aggregation for Rubric-Based Reinforcement Learning

Can Lv[1], Mingju Chen[1], Heng Chang[3], Shiji Zhou[1,2]

[1]Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing, School of Artificial Intelligence, Beihang University,
[2]Beijing Academy of Artificial Intelligence (BAAI), [3]Tsinghua University

Project Lead: Heng Chang, **Corresponding to:** Shiji Zhou <zhoushiji25@buaa.edu.cn>

Abstract

Rubric-based rewards are increasingly used for open-ended language model post-training, but criterion-level scores are often aggregated as independent utilities. This flat scalarization ignores rubric-specified prerequisite and activation relations among criteria, allowing reward or penalty to be counted even when the condition that licenses it is absent. We call this structural reward-aggregation failure **False Credit Propagation** (FCP). To address this limitation, we propose GEAR (**G**raphical **E**vent **A**ggregation for **R**ubric rewards), a probabilistic graphical framework for dependency-aware rubric aggregation. GEAR models each criterion outcome as a latent Bernoulli event in a typed rubric graph, propagates soft suppression from unsupported parent events to their children, and aggregates the resulting event probabilities into a normalized expected signed utility. This yields a linear-time reward computation that can be plugged into standard rubric-based RL pipelines without changing the outer optimization algorithm. Experiments on HealthBench, WritingBench, and PLawBench with two policy backbones show that GEAR consistently improves over flat aggregation and deterministic gating, achieving relative gains of up to 15.5% over flat aggregation. FCP diagnostics further show that GEAR reduces leakage by 96.5% relative to flat aggregation while preserving more licensed downstream utility than deterministic gating. Our code is publicly available at https://github.com/LvCan926/GEAR.

1 Introduction

Open-ended post-training of language models requires reward signals that compare candidate responses beyond exact-match correctness (Ouyang et al., 2022; Bai et al., 2022; Rafailov et al., 2024). Rubric-based rewards provide a practical interface for this setting by decomposing response quality into explicit criteria, making desiderata such as factuality, completeness, evidence use, uncertainty calibration, safety, and conversational appropriateness available to reward modeling (Arora et al., 2025; Rein et al., 2023; He et al., 2025; Jin et al., 2025). Yet decomposition alone does not define a reward: policy optimization still requires criterion-level judgments to be aggregated into a scalar objective, and this aggregation step determines what behavior the model is optimized to produce.

In this setting, a common aggregation choice is to reduce criterion-level judgments to a flat weighted sum (Gunjal et al., 2025). Although simple, this treats each criterion as an independent source of utility whose contribution is valid whenever a local judge assigns it a high score. Realistic rubrics often violate this assumption. Some criteria are licensed by prerequisites: evidence-use credit, for example, should be reduced when factual grounding fails (Starace et al., 2025). Other criteria are activated by context: a penalty for omitting medical uncertainty disclaimers should apply primarily when the response provides clinical advice (Arora et al., 2025). We call rubrics with such typed criterion dependencies **dependency-structured rubrics**. Flattening these dependencies creates a mismatch between local criterion satisfaction and rubric-level utility: locally supported criteria may be counted even when the prerequisite or contextual condition that licenses their contribution is absent.

In rubric-based reinforcement learning, this mismatch can turn local judging errors into optimization errors. We call this failure **False Credit Propagation (FCP)**: a criterion contributes reward or penalty without the rubric-level condition that licenses that contribution. FCP can occur even when each local criterion score is reasonable on its own: the local judge may correctly detect evidence use, missing caveats, or other criterion-level signals. The error arises when flat aggregation counts these

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 43980–44003
October 24-29, 2026 ©2026 Association for Computational Linguistics

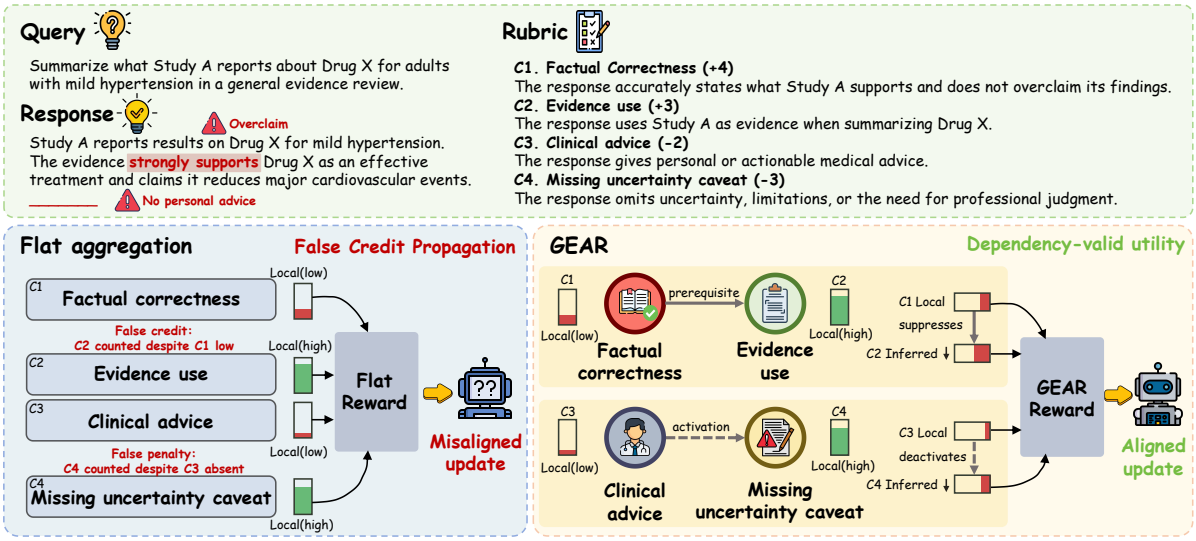


Figure 1: Overview of False Credit Propagation (FCP). Flat aggregation can propagate unsupported credit or penalties by treating rubric criteria independently. GEAR uses typed prerequisite and activation dependencies to suppress unlicensed downstream utility before reward computation.

signals without checking whether the prerequisite or activation condition that makes their utility applicable is present. As a result, unsupported credit can reinforce responses that satisfy downstream criteria only superficially, while unlicensed penalties can discourage otherwise valid responses. During policy optimization, these invalid utilities are not merely averaged away; they become part of the scalar reward signal and can therefore shape the behavior the policy learns to produce.

Addressing FCP therefore requires more than better local judges: it requires an aggregation rule that respects rubric dependencies, handles uncertainty in criterion-level judgments, and remains efficient for repeated reward computation. In particular, an RL-compatible solution must satisfy three requirements. **(1) Dependency representation.** It must represent how criteria license one another: prerequisite relations specify when downstream credit is supported by more basic conditions, while activation relations specify when a conditional criterion is applicable at all (Hong et al., 2026; Pan et al., 2026). **(2) Uncertainty-aware credit assignment.** It must suppress unlikely-to-be-licensed credit without discarding valid partial credit, since hard gating is brittle under noisy or partially calibrated criterion judges (Lambert et al., 2024; Zheng et al., 2023; Pan et al., 2026). **(3) Compatibility with repeated RL scoring.** It must add predictable low overhead during policy optimization, avoiding expensive exact inference or repeated LLM reasoning at reward time (He et al., 2025; Li et al., 2026).

These requirements suggest that FCP should be addressed at the aggregation layer, rather than by treating rubric criteria as independent checklist items. We propose **GEAR** (**G**raphical **E**vent **A**ggregation for **R**ubric rewards), a probabilistic graphical aggregation framework for dependency-structured rubric rewards (Pearl, 2014; Koller and Friedman, 2009). First, to represent rubric dependencies, GEAR constructs a typed query-specific rubric graph whose edges encode prerequisite and activation relations among criteria. Second, to handle uncertainty in criterion-level judgments, GEAR represents each criterion as a latent binary event initialized by a local judge and infers dependency-adjusted marginals with monotone conditional factors. Third, to support repeated RL scoring, GEAR aggregates these marginals into a normalized expected signed utility using a linear-time topological approximation.

Our contributions are:

- We identify and formalize **False Credit Propagation (FCP)**, a structural failure in dependency-structured rubric rewards where criteria contribute reward or penalty without the rubric-level conditions that license their utility.
- We propose GEAR, a probabilistic graphical aggregation framework that represents rubric criteria as latent events and uses typed prerequisite and activation dependencies to in-

fer dependency-adjusted criterion marginals. We further derive a linear-time topological marginal approximation for repeated RL reward computation.

- We empirically show, across multiple domain-specific benchmarks, policy backbones, and rubric-guided RL pipelines, that GEAR improves task performance while reducing dependency-level credit leakage.

2 Related Work

LLM-as-a-judge and rubric-based evaluation. Open-ended language model evaluation often lacks a single reference answer and requires multi-dimensional judgments (Zheng et al., 2023; Liu et al., 2023; Gu et al., 2025). LLM-as-a-judge methods provide a scalable interface for such judgments (Zheng et al., 2023; Liu et al., 2023; Kim et al., 2024b; Gu et al., 2025). Rubrics make this interface more interpretable by decomposing response quality into fine-grained criteria (Kim et al., 2024a,b; Hong et al., 2026), and have been used in benchmarks for medical dialogue, generative writing, explanation evaluation, and research agents (Arora et al., 2025; Wu et al., 2025; Galvan-Sosa et al., 2025; Sharma et al., 2025; Han et al., 2026). These works establish rubrics as a useful interface for criterion-level judging. GEAR addresses the subsequent aggregation problem: how should such judgments be reduced to a scalar reward when rubric criteria are semantically dependent?

Rubrics as rewards for reinforcement learning. Rubric-based rewards have recently been used to extend reinforcement learning beyond domains with automatically verifiable answers (He et al., 2025; Li et al., 2026). Rubrics as Rewards (RaR) uses instance-specific rubrics to provide reward feedback and studies strategies for converting rubric judgments into scalar training signals (Gunjal et al., 2025). Other rubric-guided RL methods use rubrics as reward anchors, rollout scaffolds, in-context reward specifications, or transition-level credit signals for long-horizon agents (Huang et al., 2025; Zhou et al., 2026; Zhang et al., 2026b,a). While these methods differ in how rubrics are generated, exposed to the policy, or used during training, GEAR focuses on the aggregation layer after criterion-level judging: it models dependencies among criteria before converting their scores into a scalar reward.

Structured rubrics and dependency-aware aggregation. Prior work has explored structured or hierarchical rubrics for evaluation and grading (Condor et al., 2022; Starace et al., 2025). More broadly, structured designs have also been explored for agent-system adaptation, lifelong memory, and decomposed task execution (Chen et al., 2026a; Lv et al., 2026; Chen et al., 2026b). For example, PaperBench decomposes AI research replication into hierarchically graded subtasks (Starace et al., 2025), while graph-based rubric representations have been studied for automatic short-answer grading (Condor et al., 2022). These works use structure to organize evaluation, but do not study how criterion dependencies should control scalar reward aggregation during policy optimization. In many rubric-reward settings, criterion scores are instead scalarized as independent checklist items, for example by summing weighted judgments (Gunjal et al., 2025; Li et al., 2026). Such flat scalarization ignores prerequisite and activation relations, and therefore cannot distinguish rubric-licensed utility from locally scored but semantically invalid utility. GEAR instead represents these relations as a query-specific typed rubric graph and defines a probabilistic graphical aggregation model over latent criterion events (Pearl, 2014; Koller and Friedman, 2009). By computing expected signed utility under dependency-adjusted criterion marginals, GEAR mitigates false credit propagation while retaining linear-time reward computation for repeated RL scoring.

3 Methodology

3.1 Problem Formulation

We study response-level reward aggregation for rubric-based reinforcement learning. Given a query x, a candidate response y, and a query-specific weighted rubric

$$\mathcal{R}_x = \{(r_{x,i}, w_{x,i})\}_{i=1}^{K_x}, \tag{1}$$

where $r_{x,i}$ is the i-th criterion and $w_{x,i} \in \mathbb{R}$ is its signed utility weight, the goal is to convert criterion-level judge scores into a scalar reward while accounting for dependencies among criteria. Positive weights reward desirable events, whereas negative weights penalize violation events. When the query and response are clear from context, we omit explicit dependence on x and y and write K, r_i, w_i, and p_i.

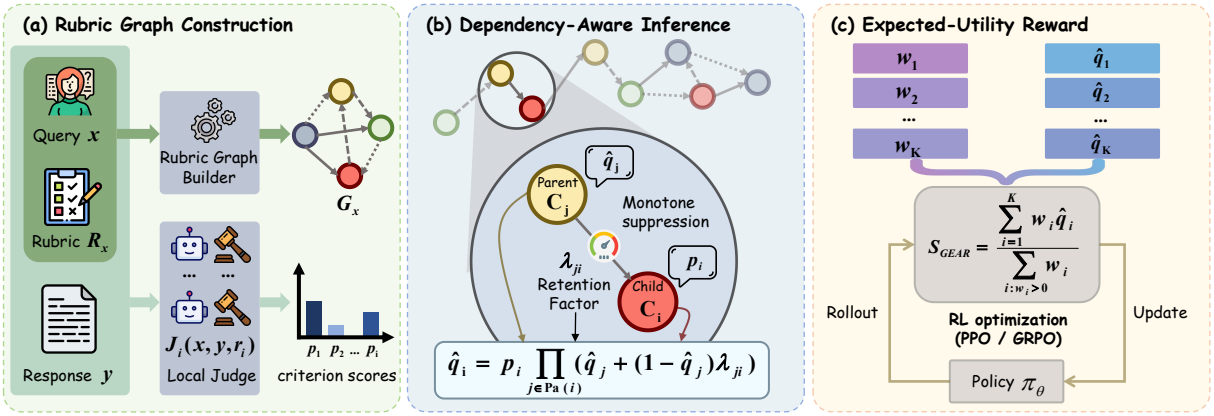


Figure 2: The GEAR framework. **(a)** A query-specific rubric graph is built from the query and rubric. **(b)** Local criterion scores are adjusted through monotone dependency-aware inference. **(c)** The inferred marginals are aggregated into a normalized expected-utility reward for policy optimization.

Flat rubric aggregation. A standard scalarization first obtains a local event score $p_i(x, y) \in [0, 1]$ for each criterion and then aggregates scores independently. Assuming positive utility mass $\sum_{i:w_i>0} w_i > 0$, the flat reward is

$$S_{\text{flat}}(x, y) = \frac{\sum_{i=1}^{K} w_i p_i(x, y)}{\sum_{i:w_i>0} w_i}. \tag{2}$$

This treats the rubric as an unordered checklist and ignores semantic dependencies among criteria.

Latent criterion events and local scores. GEAR associates each criterion r_i with a binary latent event $C_i \in \{0, 1\}$, where $C_i = 1$ means that the condition described by the criterion holds for response y. For positive-weight criteria, this event denotes desirable behavior; for negative-weight criteria, it denotes a violation or penalty condition. A local judge provides

$$p_i(x, y) = J_i(x, y, r_i) \in [0, 1], \tag{3}$$

which we interpret as a local estimate of $P(C_i = 1 \mid x, y, r_i)$. The score may be soft or binary, and the vector $\mathbf{p}(x, y) = (p_1(x, y), \ldots, p_K(x, y))$ is treated as observed input to the graphical aggregation model rather than as a variable generated by it.

3.2 Query-Specific Rubric Graph

Rubric criteria are often semantically dependent. GEAR represents these dependencies with a query-specific typed directed acyclic graph over criterion events, $\mathcal{G}_x = (\mathcal{V}_x, \mathcal{E}_x)$, where $\mathcal{V}_x = \{1, \ldots, K_x\}$ indexes rubric criteria. The graph is constructed from the query x and rubric $\mathcal{R}_x$, and is kept fixed across candidate responses y for the same query.

Typed dependency graph. Each directed edge $(j, i) \in \mathcal{E}_x$ connects a parent criterion j to a child criterion i, with edges oriented from licensing conditions to licensed criteria. We partition the edge set as

$$\mathcal{E}_x = \mathcal{E}_x^{\text{wk}} \dot{\cup} \mathcal{E}_x^{\text{st}} \dot{\cup} \mathcal{E}_x^{\text{act}}, \tag{4}$$

where $\mathcal{E}_x^{\text{wk}}$ and $\mathcal{E}_x^{\text{st}}$ denote weak and strong prerequisite edges, and $\mathcal{E}_x^{\text{act}}$ denotes activation edges. We write $\mathcal{E}_x^{\text{pre}} = \mathcal{E}_x^{\text{wk}} \cup \mathcal{E}_x^{\text{st}}$ for prerequisite edges and

$$\text{Pa}(i) = \{j : (j, i) \in \mathcal{E}_x\} \tag{5}$$

for the parents of criterion i.

Dependency semantics. Prerequisite edges encode graded support: when a prerequisite parent is unsupported, the child event remains possible but its probability should be discounted, with weak and strong prerequisites differing in suppression strength. Activation edges encode conditional applicability: a child criterion should contribute utility primarily when its activation parent holds. These licensing relations apply regardless of the child weight sign. For positive-weight children, an edge licenses downstream credit; for negative-weight children, it licenses the applicability of a violation penalty. The parent criterion contributes to the final reward only through its own utility weight, while the edge modulates the child event.

Graph construction. For each query-specific rubric, the rubric graph is constructed offline from the query and rubric, and is fixed across candidate responses. The construction procedure, criterion roles, dependency annotation process, and acyclic projection are described in Appendix A. Complete graph-annotation prompts are provided in Appendix J.1.

3.3 Monotone Graphical Reward Model

GEAR defines a conditional Bayesian network over latent criterion events, conditioned on the local score vector $\mathbf{p}$ and the query-specific rubric graph $\mathcal{G}_x$. Its CPDs are monotone: unsupported parent events can only suppress, not inflate, child-event probabilities.

Dependency retention factors. Each edge $(j, i) \in \mathcal{E}_x$ is assigned a retention factor $\lambda_{ji} \in [0, 1]$, which specifies how much child-event probability is retained when parent event C_j is unsupported. We define retention factors through a dependency-to-retention mapping

$$\lambda_{ji} = F(x, \mathcal{R}_x, \mathcal{G}_x, j, i) \in [0, 1], \tag{6}$$

where (j, i) identifies a typed parent-child edge in the rubric graph. The mapping F converts the semantic strength and type of a rubric dependency into a probabilistic retention value.

In our experiments, we use a fixed type-level instantiation of F for reproducibility: $\lambda_{ji} = \lambda_{\tau_{ji}}$, where $\tau_{ji} \in \{\mathrm{wk}, \mathrm{st}, \mathrm{act}\}$ is the edge type. We set $\lambda_{\mathrm{wk}} = 0.6$, $\lambda_{\mathrm{st}} = 0.2$, and $\lambda_{\mathrm{act}} = 0.0$ by default. Thus activation edges impose the strongest suppression, strong prerequisites intermediate suppression, and weak prerequisites the weakest suppression. Additional details on retention settings, exact-inference agreement, and sensitivity analysis are provided in Appendices B and I.

Monotone CPD. For criterion i, we define the dependency-adjusted Bernoulli parameter

$$\alpha_i(C_{\mathrm{Pa}(i)}; p_i) = p_i \prod_{j \in \mathrm{Pa}(i)} \lambda_{ji}^{1-C_j}. \tag{7}$$

Root criteria satisfy $\alpha_i = p_i$. For non-root criteria, the local score p_i is multiplicatively discounted by each unsupported parent: supported parents impose no suppression, while unsupported parents retain only a factor λ_{ji} of the child-event probability. The local CPD is

$$P_i(C_i = 1 \mid C_{\mathrm{Pa}(i)}, p_i) = \alpha_i(C_{\mathrm{Pa}(i)}; p_i). \tag{8}$$

Because $p_i, \lambda_{ji} \in [0, 1]$, this defines a valid Bernoulli CPD; moreover, the child-event probability is monotone in each parent state and bounded above by the local score p_i.

Conditional graphical model. Conditioned on $\mathbf{p}(x, y)$ and the acyclic graph $\mathcal{G}_x$, these CPDs define

$$P_{\mathcal{G}_x}(\mathbf{c} \mid \mathbf{p}) = \prod_{i=1}^{K} P_i(c_i \mid c_{\mathrm{Pa}(i)}, p_i). \tag{9}$$

This is a conditional model for aggregating externally produced judge scores, not a generative model of the scores themselves: $\mathbf{p}$ is observed input, and the graph models dependencies among latent criterion events. Since $\mathcal{G}_x$ is acyclic and every local factor is a valid CPD, Eq. 9 defines a valid distribution over $\mathbf{C}$.

3.4 Structured Reward Computation

The scalar reward is computed by first inferring dependency-adjusted criterion event probabilities and then aggregating signed utility under these marginals.

Exact marginals. For each criterion i, let

$$q_i = P_{\mathcal{G}_x}(C_i = 1 \mid \mathbf{p}) = \sum_{\mathbf{c}: c_i = 1} P_{\mathcal{G}_x}(\mathbf{c} \mid \mathbf{p}) \tag{10}$$

denote its exact marginal under the conditional graphical model in Eq. 9. Direct summation is exponential in K. Although exact variable elimination can exploit graph sparsity, repeated reward computation during policy optimization motivates a linear-time approximation.

Linear-time marginal approximation. GEAR computes approximate marginals $\hat{q}_i$ in topological order. Root criteria use their local scores, $\hat{q}_i = p_i$. For a non-root criterion, the exact marginal can be written as

$$q_i = p_i \, \mathbb{E}_{P_{\mathcal{G}_x}(\mathbf{C}_{\mathrm{Pa}(i)} \mid \mathbf{p})} \left[\prod_{j \in \mathrm{Pa}(i)} \lambda_{ji}^{1-C_j} \right]. \tag{11}$$

We approximate this parent joint moment by a product of marginal moments. Using $\hat{q}_j \approx P_{\mathcal{G}_x}(C_j = 1 \mid \mathbf{p})$ gives the topological update

$$\hat{q}_i = p_i \prod_{j \in \mathrm{Pa}(i)} \Big(\hat{q}_j + (1 - \hat{q}_j) \lambda_{ji} \Big). \tag{12}$$

This update is exact for nodes with at most one parent, and for multi-parent nodes whose parents are mutually independent under the conditional model; otherwise it is a mean-field approximation to the parent joint moment. Each parent factor lies in $[\lambda_{ji}, 1]$, approaching 1 when the parent is likely active and λ_{ji} when it is unlikely active. Hence the update smoothly suppresses unsupported downstream events and satisfies

$$p_i \prod_{j \in \mathrm{Pa}(i)} \lambda_{ji} \le \hat{q}_i \le p_i. \tag{13}$$

We evaluate exact-inference agreement on small graphs in Appendix B.2.

Method	HealthBench-500 ↑	WritingBench ↑	PLawBench ↑	Avg. ↑
Qwen2.5-7B-Instruct				
Aggregation-only				
Flat aggregation	56.7	74.1	68.4	66.4
Hard gating	54.6	72.5	65.0	64.0
GEAR	**62.9** (+6.2)	**79.6** (+5.5)	**75.2** (+6.8)	**72.6** (+6.2)
Plug-in replacement in rubric-guided RL pipelines				
RaR-Explicit	49.5	67.0	62.3	59.6
RaR-Explicit + GEAR	55.0 (+5.5)	71.6 (+4.6)	68.8 (+6.5)	65.1 (+5.5)
MeRF	40.2	61.9	55.7	52.6
MeRF + GEAR	45.6 (+5.4)	68.7 (+6.8)	60.4 (+4.7)	58.2 (+5.6)
RuscaRL	58.2	75.4	72.1	68.6
RuscaRL + GEAR	**65.7** (+7.5)	**81.5** (+6.1)	**76.9** (+4.8)	**74.7** (+6.1)
Llama-3.1-8B-Instruct				
Aggregation-only				
Flat aggregation	54.1	76.2	69.5	66.6
Hard gating	53.0	72.7	66.7	64.1
GEAR	**62.5** (+8.4)	**82.4** (+6.2)	**76.9** (+7.4)	**73.9** (+7.3)
Plug-in replacement in rubric-guided RL pipelines				
RaR-Explicit	48.4	67.9	63.8	60.0
RaR-Explicit + GEAR	54.1 (+5.7)	73.6 (+5.7)	70.5 (+6.7)	66.1 (+6.0)
MeRF	39.5	62.3	58.4	53.4
MeRF + GEAR	44.7 (+5.2)	69.8 (+7.5)	62.1 (+3.7)	58.9 (+5.5)
RuscaRL	56.9	76.7	74.2	69.3
RuscaRL + GEAR	**63.8** (+6.9)	**85.0** (+8.3)	**78.4** (+4.2)	**75.7** (+6.5)

Table 1: Main results across different benchmarks. Aggregation-only rows differ only in reward aggregation, while plug-in rows replace the aggregation step in each host pipeline. All reported scores are means over three independent runs with different random seeds. **Bold** marks the best result within each model block. Parenthesized values denote absolute gains over Flat in aggregation-only rows and over the corresponding host pipeline in plug-in rows.

Expected-utility reward. Using the approximate marginals, GEAR defines the scalar reward as normalized expected signed utility:

$$S_{\mathsf{GEAR}}(x, y) = \frac{\sum_{i=1}^{K} w_i \hat{q}_i}{\sum_{i:w_i>0} w_i}. \tag{14}$$

Because the utility is additive over criteria, only marginal event probabilities are required. Unlike flat aggregation in Eq. 2, each utility term is weighted by a dependency-adjusted event probability that incorporates both the local judge score and the rubric graph. The resulting scalar reward can be used by response-level policy optimization algorithms such as PPO and GRPO without changing the outer RL algorithm.

Limiting cases. GEAR recovers simpler aggregation rules in two limits. If all retention factors are one, then $\hat{q}_i = p_i$ for every criterion and Eq. 14 reduces to flat aggregation. If selected retention factors are zero and parent decisions are binary, the corresponding child criteria are deterministically gated by their parents, recovering hard dependency gating.

3.5 Complexity Analysis

For each response, the topological update in Eq. 12 visits every criterion node once and processes every dependency edge once. The online aggregation cost is therefore $O(K + |\mathcal{E}_x|)$ time and $O(K)$ memory for storing the marginal estimates $\hat{q}_i$; the final utility aggregation in Eq. 14 adds only an $O(K)$ pass. This cost is incurred after criterion-level judge scoring, which is shared by all aggregation variants. Graph construction is performed offline once per query-specific rubric and is fixed across candidate responses, so GEAR adds only linear-time graphical aggregation during policy optimization.

4 Experiments

4.1 Experimental Setup

Models and training. We use Qwen2.5-7B-Instruct (Qwen et al., 2025) and Llama-3.1-8B-Instruct (Grattafiori et al., 2024) as policy mod-

els, and optimize them with GRPO (Shao et al., 2024). All compared aggregation methods share the same policy model, training data, rubrics, rollout budget, and optimization pipeline; they differ only in scalar reward aggregation. We use Qwen3-8B for criterion-level training rewards and a separate Qwen3-32B judge for final validation (Yang et al., 2025). For each policy backbone and method, we run three independent training runs. Full experimental and judge settings are provided in Appendix C.

Benchmarks. We evaluate on three open-ended rubric-based benchmarks spanning different domains: HealthBench-500 for medical dialogue quality with physician-written rubrics (Arora et al., 2025), WritingBench for long-form writing with query-specific criteria (Wu et al., 2025), and PLawBench for case-based legal reasoning with problem-specific scoring rubrics (Shi et al., 2026).

Baselines. We evaluate GEAR in two settings. First, in an aggregation-only comparison, we replace only the scalarization rule within the same minimal rubric-RL pipeline and compare against **Flat**, which sums criterion scores as in Eq. 2, and **Hard**, which uses the same dependency graph but replaces soft retention with deterministic gating. Second, we test GEAR as a plug-in replacement for the aggregation step in existing rubric-guided RL pipelines, including **RaR-Explicit** (Gunjal et al., 2025), **MeRF** (Zhang et al., 2026b), and **RuscaRL** (Zhou et al., 2026).

4.2 Main Results

Table 1 shows that GEAR consistently improves rubric-based reward aggregation across benchmarks and backbones. In the aggregation-only setting, GEAR outperforms both Flat and Hard on every benchmark, raising the average score from 66.4 to 72.6 for Qwen2.5-7B-Instruct and from 66.6 to 73.9 for Llama-3.1-8B-Instruct. Hard underperforms Flat in both blocks, suggesting that deterministic gating can discard valid partial credit under noisy criterion judgments.

As a plug-in module, GEAR also improves RaR-Explicit, MeRF, and RuscaRL on both backbones. The best results are achieved by RuscaRL+GEAR, reaching average scores of 74.7 on Qwen2.5-7B-Instruct and 75.7 on Llama-3.1-8B-Instruct. This suggests that FCP is an aggregation-layer issue rather than an artifact of a particular training setup.

Method	$L_{\mathrm{FCP}} \downarrow$	$P \uparrow$
Flat aggregation	0.0922	1.0000
Hard gating	0.0000	0.1877
GEAR	0.0032	0.2706

Table 2: FCP diagnostics on edge-level dependency cases. Lower L_{FCP} indicates less invalid downstream utility under violated dependencies, while higher P indicates better preservation under satisfied dependencies.

4.3 False Credit Propagation Diagnostics

We further evaluate whether GEAR reduces False Credit Propagation (FCP) at the criterion level. The diagnostic measures a leakage–preservation tradeoff: an aggregation rule should suppress child utility when a dependency is violated.

Let $s_i^m(x, y)$ be the effective post-dependency score for criterion i under method m: $s_i^{\mathrm{flat}} = p_i$, s_i^{hard} is the hard-gated score, and $s_i^{\mathrm{gear}} = \hat{q}_i$. We define leakage cases $\mathcal{D}$ as edges $(j, i) \in \mathcal{E}_x$ where the child is locally supported but the parent is not, i.e., $p_i(x, y) \geq 0.5$ and $p_j(x, y) < 0.5$. The normalized leaked utility is

$$L_{\mathrm{FCP}}^m = \frac{1}{|\mathcal{D}|} \sum_{(x,y,j,i)\in\mathcal{D}} \frac{|w_i|}{\sum_{k:w_k>0} w_k} s_i^m(x, y). \tag{15}$$

Lower values indicate less unsupported downstream utility, including both invalid credit and invalid penalties.

To avoid rewarding methods that simply suppress all dependent criteria, we also measure preservation on satisfied dependencies. Let $\mathcal{S}$ contain edges where both parent and child are locally supported: $p_i(x, y) \geq 0.5$ and $p_j(x, y) \geq 0.5$. The preservation ratio is

$$P^m = \frac{1}{|\mathcal{S}|} \sum_{(x,y,j,i)\in\mathcal{S}} \frac{s_i^m(x, y)}{p_i(x, y)}. \tag{16}$$

Higher values indicate better preservation of dependency-licensed child utility.

Table 2 shows that GEAR reduces L_{FCP} from 0.0922 to 0.0032, a 96.5% relative reduction over Flat. Hard gating achieves slightly lower leakage, but preserves much less licensed utility than GEAR (0.1877 vs. 0.2706). Thus, GEAR keeps leakage close to zero while avoiding the over-suppression caused by hard gating. Additional details on diagnostic case construction and interpretation are provided in Appendix F.

Statistic	HealthBench
Num. rubrics	5000
Avg. criteria / rubric	11.48
Avg. edges / rubric	10.92
Avg. update size $K + \lvert\mathcal{E}_x\rvert$	22.40
Non-empty graph rate	91.84%
Invalid edge candidates / rubric	0.42
Online aggregation latency / response	4.1 ms

Table 3: Graph construction and online aggregation statistics on HealthBench, the largest rubric benchmark in our experiments. Latency measures only post-judge aggregation.

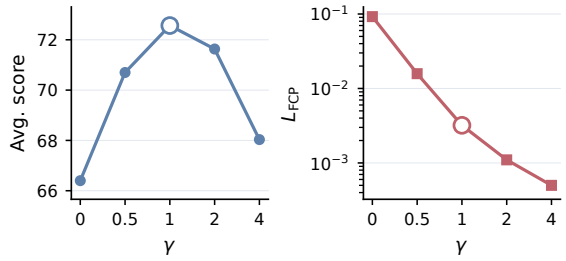


Figure 3: Sensitivity to the suppression-strength parameter γ. Larger γ imposes stronger dependency suppression; hollow points mark the default setting.

4.4 Efficiency and Construction Statistics

We evaluate the overhead of dependency-structured aggregation. Graph construction is performed offline once per query-specific rubric and fixed across candidate responses. At reward time, GEAR applies the topological update in Eq. 12, requiring $O(K + |\mathcal{E}_x|)$ time and $O(K)$ memory after local criterion scores are available.

Table 3 reports results on HealthBench. The average rubric has 11.48 criteria and 10.92 edges, with an average update size of 22.40. Online aggregation takes 4.1 ms per response, excluding generation and criterion-level judging. After validation and DAG projection, 91.84% of rubrics yield non-empty graphs, with 0.42 invalid edge candidates per rubric. Full statistics across all benchmarks are reported in Appendix G.

4.5 Sensitivity Analysis

We analyze sensitivity to the global suppression parameter γ. With the rubric graph and edge types fixed, each type-level retention factor is

$$\lambda_t(\gamma) = \begin{cases} 1, & \gamma = 0, \\ \lambda_t^{\gamma}, & \gamma > 0, \end{cases} \tag{17}$$

where $t \in \{\mathrm{wk}, \mathrm{st}, \mathrm{act}\}$ and λ_t is the default retention factor. Here, $\gamma = 0$ recovers Flat aggregation, $\gamma = 1$ gives the default GEAR, and larger values impose stronger suppression.

Variant	**Score ↑**	**Δ vs. Flat**
No graph / Flat	56.7	–
Random graph	55.8	−0.9
Untyped graph	60.4	+3.7
Full graph + hard gating	54.6	−2.1
Full graph + soft inference (GEAR)	**62.9**	+6.2

Table 4: Ablations on HealthBench-500. All variants differ only in the aggregation module; full results are reported in Appendix H.

Figure 3 shows that increasing γ sharply reduces FCP leakage. The default setting lowers L_{FCP} from 0.0922 to 0.0032 while achieving the best average benchmark score. Stronger suppression further reduces leakage but degrades task performance, indicating that moderate soft suppression gives the best score–leakage trade-off.

4.6 Ablation Studies

We ablate dependency topology, relation typing, and soft graphical inference under the same minimal rubric-RL setting as Table 1. Random graph rewires edges while preserving edge counts and type frequencies; Untyped graph uses the annotated topology with a shared retention factor; Hard gating replaces soft retention with deterministic gating.

Table 4 shows that random rewiring fails to improve over Flat, whereas the untyped graph improves from 56.7 to 60.4. The full typed graph with soft inference performs best at 62.9, while hard gating underperforms Flat. These results support meaningful topology, typed dependencies, and soft inference. Full three-benchmark ablations are reported in Appendix H.

4.7 Additional Validation and Robustness

Multi-parent aggregation. GEAR uses a product rule to combine support from multiple parents. Among dependent criteria, 70.0% have one parent, 24.4% have two parents, and only 5.6% have three or more, with an average of 1.37 parents. We additionally compare the product rule with geometric-mean, arithmetic-mean, and maximum-support aggregation under the same Qwen2.5-7B aggregation-only setup. The product rule achieves the best average score and leakage (72.6/0.0032), compared with the geometric mean (70.5/0.0058), arithmetic mean (69.4/0.0087), and maximum sup-

Method	Qwen2.5-7B	Llama-3.1-8B	Avg.
Original + Flat	66.4	66.6	66.5
Conditional Rewrite + Flat	69.9	70.4	70.2
Original + GEAR	**72.6**	**73.9**	**73.3**

Table 5: Comparison with dependency-aware rubric rewriting. Results are averaged across the three benchmarks.

Benchmark	Flat	Hard	GEAR
HealthBench	55.2	53.6	**61.5**
WritingBench	74.4	72.0	**79.8**
PLawBench	68.1	65.5	**74.4**

Table 6: Evaluation with benchmark-specific protocols. Results are averaged over both policy backbones and three seeds.

port ($68.8/0.0146$). All four dependency-aware variants outperform Flat (66.4), indicating that the gains are not unique to the product formulation, while the product rule provides the strongest score–leakage trade-off.

Rubric specification versus aggregation. GEAR distinguishes criterion omission, relational under-specification, and aggregation failure. It cannot recover an omitted criterion, but it can make an implicit licensing relation explicit and incorporate that relation into reward aggregation. To test whether textualizing the dependencies is sufficient, we construct a CONDITIONAL REWRITE baseline that inserts the same prerequisite and activation conditions into each child criterion before applying Flat aggregation. All variants use the same dependency annotations, criterion weights, judges, training budgets, and random seeds.

As shown in Table 5, conditional rewriting recovers part of the gap, confirming that rubric specification matters. GEAR provides a further improvement by retaining decomposed parent and child judgments and explicitly aggregating their dependency support.

Graph quality and human validation. We manually audit 180 retained dependency edges, with 60 edges sampled from each benchmark and all relation types represented. Two annotators independently label each edge before adjudication. The audit yields 91.7% valid-edge precision, 87.3% relation-type accuracy among valid edges, and a pooled Cohen's κ of 0.80. This audit evaluates the precision of retained edges rather than the recall of omitted dependencies.

We further annotate 300 dependency cases using human labels determined before inspecting method outputs. On these human-defined subsets, GEAR reduces violated-dependency leakage from 0.082 for Flat to 0.010, while preserving 0.648 licensed downstream utility, compared with 0.231 for Hard gating. This human-grounded audit complements the scalable mechanism-level diagnostic in Section 4.3. Finally, in a blind evaluation of 250 matched Flat–GEAR response pairs, GEAR is preferred in 72.9% of non-tied comparisons (95% Wilson CI: 66.7–78.3%), with inter-annotator agreement $\kappa = 0.72$.

Independent and cross-family evaluation. We re-evaluate the aggregation-only policies using benchmark-specific protocols: the official HealthBench setup with GPT-4.1, the released WritingBench critic, and the released PLawBench evaluation prompt with Gemini. As shown in Table 6, GEAR consistently outperforms Flat and Hard, with gains of 5.4–6.3 points over Flat.

To test model-family dependence, we also jointly replace the graph builder, training-time reward judge, and final evaluator with non-Qwen models while keeping the policy, training protocol, and random seeds fixed. GEAR retains a 5.9-point improvement over its matched Flat baseline, suggesting that the observed gains are not specific to a Qwen-centric annotation and evaluation pipeline.

5 Conclusion

We introduced False Credit Propagation (FCP), a structural failure in flat rubric aggregation where criteria contribute reward or penalty without dependencies licensing their utility. To address FCP, we proposed GEAR, a graphical aggregation framework that models criteria as latent events and uses typed prerequisite and activation relations to infer dependency-adjusted marginals. This yields linear-time reward computation for rubric-based reinforcement learning pipelines without changing the outer optimization algorithm. Across benchmarks, backbones, and rubric-guided RL pipelines, GEAR improves task performance over flat aggregation and hard gating while reducing invalid downstream utility. The results suggest that the aggregation layer itself is an important part of reward-model design, especially when rubric criteria are semantically dependent. Overall, rubric dependencies are not merely annotation structure: they should be part of the reward model, not flattened into independent checklist items.

6 Limitations

GEAR focuses on reward aggregation given criterion-level scores and a dependency-structured rubric. It models monotone prerequisite and activation relations, where an unsupported parent can only suppress a child's contribution. It does not cover compensatory, alternative, mutually exclusive, non-monotonic, or cyclic relations, nor can it recover criteria omitted from the rubric. Rubric generation and dependency-aware aggregation therefore remain complementary.

The quality of the resulting reward still depends on the local criterion judgments and the constructed graph. Incorrect or missing edges may suppress valid utility or leave false credit uncorrected. Our human audit evaluates the precision and type accuracy of retained edges, but does not measure the recall of omitted dependencies. Likewise, the scalable FCP diagnostic is based on local judge scores; the human-grounded audit provides complementary evidence but covers only a limited subset of cases.

GEAR uses fixed type-level retention factors and bounded soft judge scores, which need not be calibrated probabilities. These choices make the method simple and reproducible, but may not capture fine-grained variation across tasks or edges. In addition, the topological update becomes a mean-field approximation when multiple parent events are correlated. Learning edge-specific retention factors and using more accurate inference for dense subgraphs are promising extensions.

Finally, our experiments cover three open-ended domains, multiple policy backbones, independent evaluators, and human preference judgments, but do not exhaust possible rubric structures or deployment settings. The human evaluation measures perceived rubric alignment rather than domain-expert factual correctness. Broader expert validation and richer dependency types remain important directions for future work.

Acknowledgments

This work is supported by the National Science Foundation of China (62401327, 62672026), and the Beijing Major Science and Technology Project under Contract no. Z25110008125031. This work was supported by Beijing Academy of Artificial Intelligence (BAAI).

References

Rahul K. Arora, Jason Wei, Rebecca Soskin Hicks, Preston Bowman, Joaquin Quiñonero-Candela, Foivos Tsimpourlas, Michael Sharman, Meghan Shah, Andrea Vallone, Alex Beutel, Johannes Heidecke, and Karan Singhal. 2025. Healthbench: Evaluating large language models towards improved human health. *Preprint*, arXiv:2505.08775.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, and 32 others. 2022. Constitutional ai: Harmlessness from ai feedback. *Preprint*, arXiv:2212.08073.

Mingju Chen, Can Lv, Guibin Zhang, Heng Chang, and Shiji Zhou. 2026a. Harnessforge: Joint harness and policy evolution for adaptive agent systems. *Preprint*, arXiv:2606.01779.

Mingju Chen, Guibin Zhang, Heng Chang, Yuchen Guo, and Shiji Zhou. 2026b. A-mapreduce: Executing wide search via agentic mapreduce. *Preprint*, arXiv:2602.01331.

Aubrey Condor, Zachary Pardos, and Marcia Linn. 2022. Representing scoring rubrics as graphs for automatic short answer grading. In *Artificial Intelligence in Education*, pages 354–365, Cham. Springer International Publishing.

Diana Galvan-Sosa, Gabrielle Gaudeau, Pride Kavumba, Yunmeng Li, Hongyi Gu, Zheng Yuan, Keisuke Sakaguchi, and Paula Buttery. 2025. Rubrik's cube: Testing a new rubric for evaluating explanations on the CUBE dataset. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 23800–23839, Vienna, Austria. Association for Computational Linguistics.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, and 542 others. 2024. The llama 3 herd of models. *Preprint*, arXiv:2407.21783.

Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. 2025. A survey on llm-as-a-judge. *Preprint*, arXiv:2411.15594.

Anisha Gunjal, Anthony Wang, Elaine Lau, Vaskar Nath, Yunzhong He, Bing Liu, and Sean Hendryx. 2025. Rubrics as rewards: Reinforcement learning beyond verifiable domains. *Preprint*, arXiv:2507.17746.

Janghoon Han, Heegyu Kim, Changho Lee, Dahm Lee, Min Hyung Park, Hosung Song, Stanley Jungkyu Choi, Moontae Lee, and Honglak Lee. 2026. Deer: A benchmark for evaluating deep research agents on expert report generation. *Preprint*, arXiv:2512.17776.

Yun He, Wenzhe Li, Hejia Zhang, Songlin Li, Karishma Mandyam, Sopan Khosla, Yuanhao Xiong, Nanshu Wang, Xiaoliang Peng, Beibin Li, Shengjie Bi, Shishir G. Patil, Qi Qi, Shengyu Feng, Julian Katz-Samuels, Richard Yuanzhe Pang, Sujan Gonugondla, Hunter Lang, Yue Yu, and 6 others. 2025. Advancedif: Rubric-based benchmarking and reinforcement learning for advancing llm instruction following. *Preprint*, arXiv:2511.10507.

Yihan Hong, Huaiyuan Yao, Bolin Shen, Wanpeng Xu, Hua Wei, and Yushun Dong. 2026. Rulers: Locked rubrics and evidence-anchored scoring for robust llm evaluation. *Preprint*, arXiv:2601.08654.

Zenan Huang, Yihong Zhuang, Guoshan Lu, Zeyu Qin, Haokai Xu, Tianyu Zhao, Ru Peng, Jiaqi Hu, Zhanming Shen, Xiaomeng Hu, Xijun Gu, Peiyi Tu, Jiaxin Liu, Wenyu Chen, Yuzhuo Fu, Zhiting Fan, Yanmei Gu, Yuanyuan Wang, Zhengkai Yang, and 2 others. 2025. Reinforcement learning with rubric anchors. *Preprint*, arXiv:2508.12790.

Yongnan Jin, Xurui Li, Feng Cao, Liucun Gao, and Juanjuan Yao. 2025. Multidimensional rubric-oriented reward model learning via geometric projection reference constraints. *Preprint*, arXiv:2511.16139.

Seungone Kim, Jamin Shin, Yejin Cho, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdoo Yun, Seongjin Shin, Sungdong Kim, James Thorne, and Minjoon Seo. 2024a. Prometheus: Inducing fine-grained evaluation capability in language models. *Preprint*, arXiv:2310.08491.

Seungone Kim, Juyoung Suk, Shayne Longpre, Bill Yuchen Lin, Jamin Shin, Sean Welleck, Graham Neubig, Moontae Lee, Kyungjae Lee, and Minjoon Seo. 2024b. Prometheus 2: An open source language model specialized in evaluating other language models. *Preprint*, arXiv:2405.01535.

Daphne Koller and Nir Friedman. 2009. *Probabilistic graphical models: principles and techniques*. MIT press.

Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, Noah A. Smith, and Hannaneh Hajishirzi. 2024. Rewardbench: Evaluating reward models for language modeling. *Preprint*, arXiv:2403.13787.

Sunzhu Li, Jiale Zhao, Miteto Wei, Huimin Ren, Yang Zhou, Jingwen Yang, Shunyu Liu, Kaike Zhang, and Wei Chen. 2026. Rubrichub: A comprehensive and highly discriminative rubric dataset via automated coarse-to-fine generation. *Preprint*, arXiv:2601.08430.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-eval: Nlg evaluation using gpt-4 with better human alignment. *Preprint*, arXiv:2303.16634.

Can Lv, Heng Chang, Shengyu Tao, Mingju Chen, Zhaoxin Fan, Ziwei Zhang, Yuchen Guo, and Shiji Zhou. 2026. All-mem: Agentic lifelong memory via dynamic topology evolution. *Preprint*, arXiv:2603.19595.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. *Preprint*, arXiv:2203.02155.

Tianjun Pan, Xuan Lin, Wenyan Yang, Qianyu He, Shisong Chen, Licai Qi, Wanqing Xu, Hongwei Feng, Bo Xu, and Yanghua Xiao. 2026. Rubriceval: A rubric-level meta-evaluation benchmark for llm judges in instruction following. *Preprint*, arXiv:2603.25133.

Judea Pearl. 2014. *Probabilistic reasoning in intelligent systems: networks of plausible inference*. Elsevier.

Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, and 25 others. 2025. Qwen2.5 technical report. *Preprint*, arXiv:2412.15115.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. Direct preference optimization: Your language model is secretly a reward model. *Preprint*, arXiv:2305.18290.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2023. Gpqa: A graduate-level google-proof q&a benchmark. *Preprint*, arXiv:2311.12022.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *Preprint*, arXiv:2402.03300.

Manasi Sharma, Chen Bo Calvin Zhang, Chaithanya Bandi, Clinton Wang, Ankit Aich, Huy Nghiem, Tahseen Rabbani, Ye Htet, Brian Jang, Sumana Basu, Aishwarya Balwani, Denis Peskoff, Marcos Ayestaran, Sean M. Hendryx, Brad Kenstler, and Bing Liu. 2025. Researchrubrics: A benchmark of prompts and rubrics for evaluating deep research agents. *Preprint*, arXiv:2511.07685.

Yuzhen Shi, Huanghai Liu, Yiran Hu, Gaojie Song, Xinran Xu, Yubo Ma, Tianyi Tang, Li Zhang, Qingjing Chen, Di Feng, Wenbo Lv, Weiheng Wu, Kexin Yang, Sen Yang, Wei Wang, Rongyao Shi, Yuanyang Qiu, Yuemeng Qi, Jingwen Zhang, and 11 others. 2026. Plawbench: A rubric-based benchmark for evaluating llms in real-world legal practice. *Preprint*, arXiv:2601.16669.

Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. 2025. Paperbench: Evaluating ai's ability to replicate ai research. *Preprint*, arXiv:2504.01848.

Yuning Wu, Jiahao Mei, Ming Yan, Chenliang Li, Shaopeng Lai, Yuran Ren, Zijia Wang, Ji Zhang, Mengyue Wu, Qin Jin, and Fei Huang. 2025. Writingbench: A comprehensive benchmark for generative writing. *Preprint*, arXiv:2503.05244.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. Qwen3 technical report. *Preprint*, arXiv:2505.09388.

Huan Zhang, Mingju Chen, Dongxu Zhou, Can Lv, Heng Chang, Sen Cui, Faguo Wu, and Shiji Zhou. 2026a. Trca: Transition-wise rubric credit assignment for long-horizon llm agents. *Preprint*, arXiv:2608.16156.

Junjie Zhang, Guozheng Ma, Shunyu Liu, Haoyu Wang, Jiaxing Huang, Ting-En Lin, Fei Huang, Yongbin Li, and Dacheng Tao. 2026b. A simple "motivation" can enhance reinforcement finetuning of large reasoning models. *Preprint*, arXiv:2506.18485.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph Gonzalez, and Ion Stoica. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, pages 46595–46623. Curran Associates, Inc.

Yang Zhou, Sunzhu Li, Shunyu Liu, Wenkai Fang, Kongcheng Zhang, Jiale Zhao, Jingwen Yang, Yihe Zhou, Jianwei Lv, Tongya Zheng, Hengtong Lu, Wei Chen, Yan Xie, and Mingli Song. 2026. Breaking the exploration bottleneck: Rubric-scaffolded reinforcement learning for general llm reasoning. *Preprint*, arXiv:2508.16949.

Appendix

A Rubric Graph Construction

This appendix describes how query-specific rubric graphs are constructed for GEAR. The graph construction procedure instantiates the typed dependency graph introduced in Section 3.2. For each query-specific rubric, the graph builder receives the query, optional source metadata, and the list of rubric criteria with their signed point values. It outputs a directed acyclic graph whose nodes are rubric criteria and whose edges represent dependency relations among them.

Graph construction is performed offline before reinforcement-learning training. For all benchmarks, we use Qwen3-8B as the graph annotation model. The same construction pipeline is applied to HealthBench, WritingBench, and PLawBench. The benchmarks use domain-specific annotation instructions, but share the same criterion roles, dependency types, validation constraints, and acyclicity requirement.

The graph is constructed only from the query and the rubric, not from any candidate response. Therefore, the resulting graph is fixed across all candidate responses for the same query and cannot leak response-specific information into reward computation. Complete graph-annotation prompts are provided in Appendix J.1.

A.1 Construction Overview

Graph construction proceeds in four stages. First, each rubric criterion is assigned a coarse functional role. Second, these roles are used to form a restricted set of candidate directed parent–child pairs. Third, the graph annotation model determines whether each candidate pair expresses a dependency relation. Finally, the resulting graph is validated and projected to a directed acyclic graph before reward aggregation.

The final reward model uses only the typed dependency edges. The criterion roles are used to guide candidate generation and to prevent implausible dependencies, such as treating an optional refinement as a prerequisite for core task correctness.

A.2 Criterion Roles

Each rubric criterion is assigned one of four functional roles: core criterion, additional criterion, penalty criterion, or applicability criterion.

A core criterion captures a primary requirement of the response, such as correctness, safety, responsiveness, completeness, task fulfillment, legal reasoning, or factual grounding. An additional criterion captures a helpful refinement that can improve the response but is not central to satisfying the task. A penalty criterion captures an undesirable error, omission, violation, or unsafe statement that should reduce reward when present. An applicability criterion captures a context-specific condition that determines whether another criterion is relevant.

The role annotation is intended to capture the functional role of a criterion within the rubric, rather than its surface wording. For example, a criterion requiring accurate use of evidence is usually a core criterion. A criterion penalizing an unsafe recommendation is a penalty criterion. A criterion indicating that the user is asking for personal medical advice may serve as an applicability criterion for downstream safety penalties.

A.3 Candidate Dependency Generation

After assigning criterion roles, we generate candidate directed dependencies using simple role-based constraints. Self-dependencies are excluded.

Core criteria may serve as prerequisites for other core criteria, additional criteria, or penalty criteria. This captures cases where a downstream criterion is meaningful only after a more basic requirement has been satisfied. For example, credit for nuanced evidence use may depend on the response first stating the relevant evidence correctly.

Applicability criteria may activate additional criteria or penalty criteria. This captures conditional rubric items whose relevance depends on the context. For example, a medical uncertainty disclaimer may be especially relevant when the response provides actionable clinical advice.

Other role combinations are not proposed as candidate dependencies. In particular, additional criteria and penalty criteria are not used as prerequisites for core criteria. This keeps the candidate graph aligned with the intended direction of rubric licensing, from more basic or contextual conditions to criteria whose utility depends on those conditions.

A.4 Dependency Annotation

For each candidate directed pair, the graph annotation model determines whether the parent criterion licenses the child criterion. When a dependency is present, it is assigned one of three relation types.

A weak prerequisite relation means that the child criterion is supported by the parent, but can still receive partial credit or penalty independently. A strong prerequisite relation means that the child criterion depends substantially on the parent and should contribute little utility when the parent is unsupported. An activation relation means that the parent controls whether a context-specific child criterion is applicable.

If the two criteria are independent or merely topically related, no dependency is added. The annotator is instructed not to add edges solely because two criteria discuss similar content. A dependency is added only when the child criterion clearly depends on, specializes, or is activated by the parent criterion.

A.5 Graph Validation and Acyclic Projection

After dependency annotation, the graph is validated before being used for reward aggregation. The validation step ensures that all edges connect valid rubric criteria, obey the role-based candidate constraints, and do not contain self-dependencies or duplicates.

The validated graph is then projected to a directed acyclic graph. This is necessary because the inference procedure in Section 3.4 processes criteria in topological order. If the annotated dependencies contain cycles, the graph builder removes redundant or lower-priority dependencies until the graph becomes acyclic. This projection is performed once before training, and the resulting graph is kept fixed for all candidate responses associated with the same query.

A.6 Benchmark-Specific Annotation Profiles

We use benchmark-specific annotation profiles to adapt the same graph construction procedure to different domains. The profiles differ only in the natural-language instructions given to the graph annotation model. They share the same criterion roles, dependency types, validation constraints, and acyclicity requirement.

For HealthBench, the annotation profile emphasizes medical correctness, safety, responsiveness, and context-specific clinical conditions. For WritingBench, it emphasizes task fulfillment, content coverage, factual grounding, structure, format compliance, audience fit, style compliance, and length compliance. For PLawBench, it emphasizes legal conclusions, fact summaries, legal reasoning, statutory basis, legal authorities, and undesirable legal errors.

Across all benchmarks, the annotator is instructed to avoid dependencies based on mere topical overlap. A dependency is included only when it reflects a clear licensing relation between rubric criteria. Complete benchmark-specific graph annotation prompts are listed in Appendix J.1.

Relation type	Retention factor	Interpretation
Weak prerequisite	$\lambda_{\mathrm{wk}} = 0.6$	Mild suppression
Strong prerequisite	$\lambda_{\mathrm{st}} = 0.2$	Strong suppression
Activation	$\lambda_{\mathrm{act}} = 0.0$	Full deactivation

Table 7: Default retention factors used by GEAR. A lower value means that the child criterion is more strongly suppressed when the parent criterion is unsupported.

B Inference Approximation and Retention Settings

This appendix reports additional details for the inference approximation used by GEAR. The main text defines the exact graphical marginal and the linear-time topological approximation. Here, we specify the default retention factors and report an exact-inference agreement check on small rubric graphs.

B.1 Default Retention Factors

GEAR uses fixed type-level retention factors in the main experiments. The same values are used across benchmarks, policy backbones, and aggregation settings. Table 7 reports the default values.

These values encode the intended ordering among dependency types. Weak prerequisites allow substantial downstream utility to remain when the parent is uncertain or unsupported. Strong prerequisites retain only a small fraction of the child utility when the parent is unsupported. Activation edges fully deactivate conditional child criteria when the activating condition is absent.

The retention factors are not tuned separately for each benchmark. This fixed setting isolates the effect of dependency-aware aggregation from learning or selecting edge-specific strengths. Sensitivity to the global suppression-strength parameter is reported in Appendix I.

B.2 Exact-Inference Agreement

The topological update used by GEAR is a linear-time approximation to the exact graphical marginal.

Benchmark	Graph-response pairs	Avg. criteria	Marginal MAE	Reward MAE	Reward correlation
HealthBench	5,000	11.48	0.0068	0.0042	0.9968
WritingBench	1,000	5.00	0.0027	0.0018	0.9992
PLawBench	250	4.00	0.0019	0.0013	0.9995

Table 8: Agreement between exact graphical inference and the linear-time topological approximation. Lower marginal and reward errors indicate closer agreement; higher reward correlation indicates closer ranking agreement across graph-response pairs.

To check whether this approximation changes the reward substantially in the small-graph regime, we compare approximate inference against exact enumeration on the rubric graphs used in our experiments.

For each evaluated graph-response pair, we compute both exact criterion marginals and topological approximate marginals using the same local criterion scores, signed rubric weights, graph structure, and retention factors. We then compare the two inference modes at both the criterion-marginal level and the final scalar-reward level.

Table 8 reports the agreement results. The marginal error measures the average absolute difference between exact and approximate criterion marginals. The reward error measures the absolute difference between the scalar rewards computed from exact and approximate marginals. The reward correlation measures agreement across evaluated graph-response pairs.

The agreement check shows that the topological approximation closely matches exact inference in the evaluated rubric graphs. The approximation error is largest on HealthBench, which has larger rubrics and denser dependency graphs, but the scalar reward remains highly correlated with the exact-inference reward. The smaller WritingBench and PLawBench graphs show lower marginal and reward errors.

This check is used only to validate the approximation. The main reinforcement-learning experiments use the linear-time topological update so that reward aggregation remains efficient for repeated policy optimization.

C Experimental Protocol

This appendix provides the experimental details needed to reproduce the main results. Across aggregation-only experiments, all methods use the same policy backbone, training data, rubrics, criterion-level reward judge, rollout budget, and reinforcement-learning hyperparameters. They differ only in the scalar reward used for policy optimization. In the plug-in experiments, GEAR replaces the reward aggregation component of each host rubric-guided reinforcement-learning pipeline, while the remaining training and evaluation setup is kept unchanged.

Benchmark	Train	Validation	Total
HealthBench	4,500	500	5,000
WritingBench	900	100	1,000
PLawBench	200	50	250

Table 9: Dataset splits used in the experiments.

C.1 Datasets and Splits

We evaluate on three rubric-based open-ended benchmarks. For HealthBench, we use 4,500 examples for training and HealthBench-500 as the held-out validation set. For WritingBench, we use 900 examples for training and 100 examples for validation. For PLawBench, we use the *practical case analysis* subset, which contains case-analysis questions together with reference answers, scoring rubrics, and score sheets. We split this subset into 200 training examples and 50 validation examples.

All rubrics are provided by the corresponding benchmark datasets. We do not generate additional rubrics for the main experiments. For each query-specific rubric, the dependency graph is constructed once before training and is kept fixed across all candidate responses for the same query.

C.2 Policy Models

We use Qwen2.5-7B-Instruct and Llama-3.1-8B-Instruct as policy backbones. Both backbones are trained under the same reinforcement-learning configuration. Unless otherwise specified, all aggregation variants for the same backbone share the same training data, rollout procedure, judge configuration, and optimization hyperparameters.

Configuration	Value	Configuration	Value
RL algorithm	GRPO	Training framework	VERL
Training hardware	7×8 Ascend 910B NPUs	Training batch size	112
PPO mini-batch size	56	Rollout samples per prompt	8
Responses per training step	896	Total training steps	350
Total epochs	5	Learning rate	1×10^{-6}
Maximum prompt length	4096	Maximum response length	4096
Rollout maximum model length	8192	Rollout maximum batched tokens	16384
Warmup schedule	Constant	KL loss coefficient	1×10^{-3}
KL loss type	Low-variance KL	Entropy coefficient	0
Tensor parallel size	4	Rollout memory utilization	0.6
Sampling temperature	0.7	Top-p	0.8
Top-k	20	Fine-tuning type	LoRA
LoRA rank	16	LoRA alpha	32

Table 10: Main GRPO training configuration. All aggregation variants use the same configuration; only the reward signal differs.

C.3 Training Configuration

All policies are optimized with GRPO using the VERL training framework. Training is performed on Ascend 910B NPUs. The main hyperparameters are reported in Table 10.

C.4 Repeated Runs

Each main result is averaged over three independent runs. The repeated runs account for stochasticity in rollout sampling, data ordering, and policy optimization. We do not tune aggregation-specific hyperparameters separately for different runs.

The main text reports mean scores for readability. Standard deviations and additional result tables are reported in Appendix E.

C.5 Plug-in Replacement Protocol

For plug-in experiments, we evaluate GEAR as a replacement for the reward aggregation step in existing rubric-guided reinforcement-learning pipelines. Specifically, for RaR-Explicit, MeRF, and RuscaRL, we keep the host pipeline unchanged and replace only the scalar reward construction from criterion-level rubric scores.

For each host pipeline and its corresponding GEAR variant, the policy backbone, training split, rubric set, criterion-level reward judge, rollout budget, and optimization hyperparameters are kept the same. This comparison isolates the effect of dependency-aware reward aggregation from other aspects of the training pipeline.

C.6 Implementation and Compute

During training, one worker node is used to serve the reward judge, and seven worker nodes are used for reinforcement-learning optimization. Each reinforcement-learning worker uses eight Ascend 910B NPUs. The maximum number of concurrent reward-judge workers is set to 32.

Benchmark	Local criterion score
HealthBench	Satisfaction probability from the binary/probability rubric judge.
WritingBench	Normalized score from the scale-based rubric-scoring wrapper.
PLawBench	Normalized score from the partial-credit or subitem-level scoring wrapper.

Table 11: Benchmark-specific criterion-level scoring. All scores are normalized to $[0, 1]$ before reward aggregation.

Criterion-level judge calls are batched over rubric items for efficiency. Each judge call evaluates at most four rubric criteria for one candidate response. The maximum judge output length is 1024 tokens, and the request timeout is 300 seconds.

Graph construction is performed offline once for each query-specific rubric. During policy optimization, the trained policy only uses the fixed graph and the criterion-level judge scores associated with each candidate response.

D Judge Models and Evaluation Protocol

This appendix describes the criterion-level judging and final evaluation protocol used in our experiments. The purpose of this protocol is to separate three components: criterion-level judging, reward aggregation, and final evaluation. All compared aggregation methods receive the same criterion-level judge outputs for a given query-response pair.

Setting	Value
Reward judge	Qwen3-8B
Evaluation judge	Qwen3-32B
Temperature	0.0
Top-p	0.8
Top-k	20
Maximum output length	1024 tokens
Request timeout	300 seconds
Maximum concurrent workers	32
Maximum rubric items per request	4

Table 12: Judge inference configuration used for reward judging and final evaluation.

Therefore, differences among Flat aggregation, deterministic gating, and GEAR are attributable to the reward signal used during policy optimization, rather than to different judge calls.

D.1 Criterion-Level Reward Judge

During reinforcement-learning training, we use Qwen3-8B as the criterion-level reward judge. For each candidate response, the judge receives the input conversation, the candidate assistant response, and the rubric items associated with the query. It then evaluates the candidate response against the rubric criteria.

For positive-weight criteria, a high local score indicates that the response satisfies the desirable criterion. For negative-weight criteria, a high local score indicates that the undesirable condition described by the criterion is present. Thus, all criterion-level scores are interpreted as event probabilities: they estimate whether the event described by the rubric item holds for the candidate response.

Each judge request evaluates one candidate response against a small batch of rubric items. In the main experiments, each request contains at most four rubric items. This batching is used only for efficiency; all aggregation methods consume the same resulting criterion-level scores.

D.2 Local Criterion Scores

All local criterion scores are normalized to the interval $[0, 1]$ before reward aggregation. The normalization procedure depends on the scoring format of the benchmark.

For HealthBench, the reward judge produces both a binary decision and a satisfaction probability for each rubric item. We use the satisfaction probability as the local criterion score. The binary decision is used for validation and for deterministic gating baselines.

For WritingBench, rubric items are scored with a normalized scale-based scoring wrapper. The raw score assigned by the judge is converted to a value in $[0, 1]$ before aggregation.

For PLawBench, rubric items are scored with a partial-credit scoring wrapper. When a rubric item contains explicit subitems, the judge evaluates the relevant subitems and the wrapper converts the awarded credit into a normalized criterion-level score.

D.3 Final Evaluation Judge

For final validation, we use Qwen3-32B as the evaluation judge. The evaluation judge is separate from the Qwen3-8B reward judge used during training. It is queried only after policy optimization and is never used for reward computation, rollout selection, or gradient updates.

For each trained policy, we generate responses on the validation split and evaluate them with Qwen3-32B under the corresponding benchmark rubric. The evaluator is not given the identity of the training method. The reported benchmark score is the mean normalized rubric score over validation examples. All scores in the main text are reported on a percentage scale.

This separation reduces the risk that the policy is evaluated by the exact same model that produced the training reward. It also makes the comparison more conservative: the policies are optimized using Qwen3-8B reward signals, while final reported performance is measured by a stronger and separate evaluator.

D.4 Judge Decoding and Batching

Both reward judging and final evaluation use deterministic decoding for rubric scoring. The judge is instructed to return compact machine-readable judgments without free-form explanations. The same decoding and batching configuration is used across aggregation variants.

D.5 Parsing and Failure Handling

The judge output is parsed into criterion-level decisions and scores. Parsed scores are clipped to the valid scoring range and then normalized to $[0, 1]$ when necessary. For binary fields, common affirmative and negative variants are mapped to Boolean decisions.

Malformed or incomplete judge outputs are not treated as successful positive judgments. When strict failure handling is enabled, invalid outputs

Method	HealthBench-500 ↑	WritingBench ↑	PLawBench ↑	Avg. ↑
Qwen2.5-7B-Instruct				
Flat aggregation	56.7 ± 0.6	74.1 ± 0.4	68.4 ± 0.7	66.4 ± 0.5
Hard gating	54.6 ± 0.8	72.5 ± 0.5	65.0 ± 0.9	64.0 ± 0.6
GEAR	$\mathbf{62.9 \pm 0.5}$	$\mathbf{79.6 \pm 0.4}$	$\mathbf{75.2 \pm 0.6}$	$\mathbf{72.6 \pm 0.4}$
Llama-3.1-8B-Instruct				
Flat aggregation	54.1 ± 0.7	76.2 ± 0.6	69.5 ± 0.8	66.6 ± 0.6
Hard gating	53.0 ± 0.6	72.7 ± 0.7	66.7 ± 0.9	64.1 ± 0.7
GEAR	$\mathbf{62.5 \pm 0.6}$	$\mathbf{82.4 \pm 0.5}$	$\mathbf{76.9 \pm 0.7}$	$\mathbf{73.9 \pm 0.5}$

Table 13: Aggregation-only validation results with standard deviations over three independent runs. All rows within each model block share the same training and evaluation setup and differ only in reward aggregation. **Bold** marks the best result within each model block.

Method	HealthBench-500 ↑	WritingBench ↑	PLawBench ↑	Avg. ↑
Qwen2.5-7B-Instruct				
RaR-Explicit	49.5 ± 0.7	67.0 ± 0.5	62.3 ± 0.8	59.6 ± 0.6
RaR-Explicit + GEAR	55.0 ± 0.6	71.6 ± 0.5	68.8 ± 0.7	65.1 ± 0.5
MeRF	40.2 ± 0.9	61.9 ± 0.7	55.7 ± 0.8	52.6 ± 0.7
MeRF + GEAR	45.6 ± 0.8	68.7 ± 0.6	60.4 ± 0.9	58.2 ± 0.6
RuscaRL	58.2 ± 0.6	75.4 ± 0.5	72.1 ± 0.7	68.6 ± 0.5
RuscaRL + GEAR	$\mathbf{65.7 \pm 0.5}$	$\mathbf{81.5 \pm 0.4}$	$\mathbf{76.9 \pm 0.6}$	$\mathbf{74.7 \pm 0.4}$
Llama-3.1-8B-Instruct				
RaR-Explicit	48.4 ± 0.8	67.9 ± 0.6	63.8 ± 0.8	60.0 ± 0.6
RaR-Explicit + GEAR	54.1 ± 0.6	73.6 ± 0.5	70.5 ± 0.7	66.1 ± 0.5
MeRF	39.5 ± 1.0	62.3 ± 0.8	58.4 ± 0.9	53.4 ± 0.8
MeRF + GEAR	44.7 ± 0.8	69.8 ± 0.6	62.1 ± 0.9	58.9 ± 0.7
RuscaRL	56.9 ± 0.7	76.7 ± 0.6	74.2 ± 0.7	69.3 ± 0.6
RuscaRL + GEAR	$\mathbf{63.8 \pm 0.6}$	$\mathbf{85.0 \pm 0.5}$	$\mathbf{78.4 \pm 0.6}$	$\mathbf{75.7 \pm 0.5}$

Table 14: Plug-in replacement validation results with standard deviations over three independent runs. For each host pipeline, GEAR replaces only the reward aggregation step. **Bold** marks the best result within each model block.

stop the run for debugging. For scale-based and partial-credit scoring wrappers, missing judgments are assigned conservative zero-score fallbacks unless strict failure handling is enabled. This prevents parsing failures from being converted into unsupported positive reward.

D.6 Reward–Evaluation Separation

The reward judge and evaluation judge play distinct roles. Qwen3-8B provides criterion-level reward signals during reinforcement-learning training. Qwen3-32B evaluates the final trained policies on held-out validation examples.

The evaluation judge is not used during training. Its scores do not affect reward computation, candidate response selection, or policy updates. Consequently, the final validation scores measure generalization to a separate judge rather than direct optimization against the training-time reward judge.

Complete judge prompt templates are provided in Appendix J.2.

E Additional Results and Statistical Reporting

This appendix reports additional numerical results omitted from the main text for space. The main result table reports mean validation scores over three independent runs. Here, we provide the corresponding standard deviations for both the aggregation-only comparison and the plug-in replacement comparison.

E.1 Aggregation-Only Results with Standard Deviations

Table 13 reports aggregation-only validation results with standard deviations over three independent runs. Within each policy backbone, all methods use the same training and evaluation setup; only

Method	Health. ↑	Writing ↑	PLaw ↑	Avg. ↑
Flat	58.4	77.8	71.2	69.1
Hard	56.5	75.4	68.1	66.7
GEAR	**65.1** (+6.7)	**84.0** (+6.2)	**78.3** (+7.1)	**75.8** (+6.7)

Table 15: Additional aggregation-only results with Qwen3-30B-A3B-Instruct as the policy backbone. Scores are from a single training run. Parenthesized values denote absolute gains over Flat aggregation.

the reward aggregation rule differs.

E.2 Plug-in Replacement Results with Standard Deviations

Table 14 reports plug-in replacement results with standard deviations over three independent runs. For each host pipeline, GEAR replaces only the reward aggregation step. The policy backbone, training split, rubric set, criterion-level reward judge, rollout budget, and optimization hyperparameters are kept fixed between the host pipeline and its corresponding GEAR variant.

E.3 Additional Backbone Results

Table 15 reports additional aggregation-only results with Qwen3-30B-A3B-Instruct as the policy backbone. The experimental setup follows the aggregation-only protocol in Appendix C. All methods use the same training data, rubrics, criterion-level reward judge, rollout budget, and optimization hyperparameters; only the reward aggregation rule differs. Since these experiments are intended as an additional backbone check, we report single-run validation scores.

F Additional Details for FCP Diagnostics

This appendix provides implementation details for the false credit propagation diagnostic in Section 4.3. The leakage and preservation metrics are defined in the main text.

For each validation response, we evaluate dependency cases over the query-specific rubric graph used by the reward aggregator. A directed dependency from parent criterion j to child criterion i is treated as violated when the child is locally supported but the parent is not, i.e., $p_i \geq 0.5$ and $p_j < 0.5$. It is treated as satisfied when both the parent and child are locally supported, i.e., $p_i \geq 0.5$ and $p_j \geq 0.5$. We use the same threshold of 0.5 across all benchmarks. The threshold is used to construct the diagnostic subsets, while the aggregation methods operate on the original local criterion scores.

The diagnostic includes both prerequisite and activation relations. Prerequisite relations test whether downstream utility is suppressed when a supporting condition is absent. Activation relations test whether conditional bonuses or penalties are suppressed when the condition that makes them applicable is absent. For negative-weight child criteria, remaining downstream utility is counted as leakage in the same way as for positive-weight criteria, because an unlicensed penalty can also distort the scalar reward.

We report both leakage under violated dependencies and preservation under satisfied dependencies. This prevents an aggregation rule from appearing effective merely by suppressing all dependent criteria. In particular, deterministic gating can eliminate threshold-defined leakage by construction while preserving substantially less licensed downstream utility.

The diagnostic should be interpreted as an aggregation-level probe rather than a human semantic audit. It relies on the same local judge scores and rubric graphs used for reward computation. Therefore, it measures how an aggregation rule behaves under the provided criterion-level evidence, rather than whether every dependency annotation or criterion judgment is semantically correct.

G Efficiency and Graph Statistics

This appendix provides additional graph-construction and aggregation-efficiency statistics. The main text reports the HealthBench statistics because it is the largest benchmark in our experiments. Here, we report the corresponding statistics for all three benchmarks.

Graph construction is performed offline once for each query-specific rubric. The online cost reported here refers only to the post-judge aggregation step: local criterion scores have already been produced by the reward judge. Therefore, the latency numbers do not include model generation or criterion-level judge inference.

G.1 Graph Construction Statistics

Table 16 reports graph statistics over the full training and validation splits. HealthBench has the largest rubrics and the densest dependency graphs, with an average of 11.48 criteria and 10.92 retained edges per rubric. WritingBench and PLawBench have smaller rubrics and correspondingly smaller

Statistic	HealthBench	WritingBench	PLawBench
Number of rubrics	5,000	1,000	250
Average criteria per rubric	11.48	5.00	4.00
Average candidate edges per rubric	21.36	5.74	3.82
Average retained edges per rubric	10.92	2.86	1.94
Average update size	22.40	7.86	5.94
Non-empty graph rate	91.84%	78.60%	71.20%
Invalid edge candidates per rubric	0.42	0.18	0.11
Post-judge aggregation latency	4.1 ms	1.7 ms	1.3 ms

Table 16: Graph construction and online aggregation statistics over the full training and validation splits. The update size is the number of criteria plus the number of retained dependency edges. Latency measures only post-judge aggregation and excludes criterion-level judge inference.

Variant	HealthBench-500	WritingBench	PLawBench	Avg.	Δ vs. Flat
Flat aggregation	56.7	74.1	68.4	66.4	–
Random graph	55.8	73.4	67.6	65.6	-0.8
Untyped graph	60.4	77.5	72.0	70.0	$+3.6$
Typed graph + deterministic gating	54.6	72.5	65.0	64.0	-2.4
Typed graph + soft inference (GEAR)	**62.9**	**79.6**	**75.2**	**72.6**	**+6.2**

Table 17: Full ablation results across all benchmarks. All variants use the same training and evaluation setup and differ only in the aggregation module.

graphs.

The non-empty graph rate measures the fraction of rubrics for which at least one dependency edge remains after validation and acyclic projection. Invalid edge candidates are edge proposals removed by the validation procedure, for example because they violate the role-based candidate constraints or do not connect valid rubric criteria.

G.2 Online Aggregation Overhead

The online aggregation step is small relative to response generation and criterion-level judging. For each response, the aggregator processes the rubric criteria and retained dependency edges once. The resulting latency is 4.1 ms per response on HealthBench, 1.7 ms on WritingBench, and 1.3 ms on PLawBench.

These results indicate that dependency-aware aggregation adds negligible post-judge overhead in the evaluated settings. The main computational cost remains criterion-level judging, which is shared by all compared aggregation methods.

H Ablation Studies

This appendix provides the full ablation results for the components of GEAR. The goal is to isolate the contribution of dependency topology, relation typing, and soft dependency-aware inference. All variants use the same policy backbone, training data, rubrics, criterion-level reward judge, rollout budget, and optimization configuration. They differ only in the graph structure or dependency-handling mechanism used during reward aggregation.

H.1 Graph Structure and Dependency Handling

Table 17 reports ablation results across all three benchmarks. The flat variant removes the rubric graph and treats all criteria as independent. The random-graph variant preserves the number of edges and relation-type frequencies, but rewires dependencies between criteria. The untyped-graph variant keeps the annotated graph topology but replaces relation-specific retention factors with a shared retention factor. The deterministic-gating variant keeps the full typed graph but applies dependencies as all-or-nothing masks. The full GEAR variant uses the annotated typed graph with soft dependency-aware inference.

The random-graph variant tests whether the gain comes merely from suppressing some downstream criteria. If random rewiring fails to improve over flat aggregation, the result indicates that arbitrary suppression is not sufficient. The untyped-graph variant tests whether annotated topology alone is useful. The comparison between the untyped graph and the full typed graph measures the effect of relation typing. The deterministic-gating variant

γ	Avg. score	L_{FCP}
0	66.4	0.0922
0.5	70.7	0.0158
1	**72.6**	0.0032
2	71.6	0.0011
4	68.0	**0.0005**

Table 18: Sensitivity to the suppression-strength parameter. The average score is computed across the evaluated benchmarks. Lower L_{FCP} indicates less false credit propagation.

tests whether hard dependency enforcement is sufficient. The full model performs best when meaningful topology, relation typing, and soft dependency-aware inference are used together.

I Sensitivity Analysis

This appendix provides additional details for the sensitivity analysis in Section 4.5. The goal is to test whether GEAR depends heavily on a particular suppression strength. We keep the rubric graph and relation types fixed, and vary only the global suppression-strength parameter used to adjust retention factors.

I.1 Retention Strength

Table 18 reports the sensitivity results. The setting $\gamma = 0$ corresponds to flat aggregation because dependency suppression is removed. The setting $\gamma = 1$ is the default configuration. Larger values impose stronger dependency suppression.

The results show a score–leakage trade-off. Increasing suppression strength consistently reduces false credit propagation, but overly strong suppression lowers validation performance. The default setting achieves the best average score while substantially reducing leakage relative to flat aggregation.

J Prompt Templates

This appendix provides the prompt templates used for graph annotation and rubric-level judging. The templates are included for reproducibility. During experiments, benchmark-specific information, rubric items, candidate pairs, and candidate responses are inserted into these templates.

J.1 Graph Annotation Prompts

Graph annotation uses two types of prompts. The node prompt assigns a functional role to each rubric criterion. The edge prompt determines whether a directed candidate pair expresses a dependency relation. All graph annotation prompts require compact JSON output so that the resulting annotations can be validated before reward aggregation.

HealthBench Node Annotation Prompt

```
You are annotating HealthBench rubrics for structured
      reward aggregation.

Task:
Classify each rubric item into exactly one criterion role.

Allowed criterion roles:
- foundation
- bonus
- penalty
- activation

Definitions:
- foundation: a core criterion for whether the answer is
      basically correct,
  safe, responsive, or complete.
- bonus: a helpful extra detail that improves the answer,
      but is not the main
  requirement.
- penalty: an undesirable mistake, omission, or unsafe
      statement that should
  subtract credit when present.
- activation: a context condition whose main role is to
      determine whether
  another rubric item is applicable.

Few-shot examples:
- "States that potassium 3.9 mmol/L is normal." ->
      foundation
- "Adds that warm compresses may help a likely muscle
      strain." -> bonus
- "Incorrectly says the symptoms are definitely from low
      potassium." -> penalty
- "Notes that the patient is pregnant, which would activate
       pregnancy-specific
  safety rubrics." -> activation

Return JSON only with this schema:
{
  "nodes": [
    {"id": "r1", "role": "foundation"}
  ]
}

Rules:
- Use only the provided rubric ids.
- Do not modify rubric text.
- Every rubric id must appear exactly once.
- Prefer foundation over bonus only when the criterion is
      truly core.
- Use activation only when the rubric mainly serves as an
      applicability
  condition for another rubric item.
```

HealthBench Edge Annotation Prompt

```
You are annotating directed rubric dependencies for
      structured reward
aggregation.

Task:
For each candidate directed pair, decide whether the parent
       criterion licenses
the child criterion.

Allowed labels:
- none
- weak prerequisite
- strong prerequisite
- activation

Definitions:
```

```
- weak prerequisite: the child is meaningfully supported by
      the parent and
  usually makes more sense if the parent is satisfied, but
      the dependency is not
  absolute.
- strong prerequisite: the child strongly depends on the
      parent; if the parent
  is absent, the child should usually receive little or no
      credit or penalty.
- activation: the parent determines whether a context-
      specific child bonus or
  child penalty is applicable.
- none: the two rubrics are largely independent.

Few-shot examples:
- Parent: "Explains that potassium 3.9 mmol/L is normal."
  Child: "Concludes that low potassium is unlikely to be
      the main cause."
  Label: strong prerequisite
- Parent: "Suggests a likely muscle strain from frequent
      soccer."
  Child: "Adds that warm compresses may help."
  Label: weak prerequisite
- Parent: "Mentions the patient is pregnant."
  Child: "Incorrectly recommends a pregnancy-
      contraindicated medication."
  Label: activation
- Parent: "Recommends colon cancer screening starting now."
  Child: "Mentions using warm compresses for leg cramps."
  Label: none

Return JSON only with this schema:
{
  "edges": [
    {"parent": "r1", "child": "r2", "relation": "weak
      prerequisite"}
  ]
}

Rules:
- Use only the provided candidate pairs.
- Return exactly one decision for each candidate pair.
- Use "none" only when there is no meaningful dependency.
- Add a dependency only when the child clearly depends on,
      specializes, or is
  activated by the parent.
```

WritingBench Node Annotation Prompt

```
You are annotating WritingBench rubrics for structured
      reward aggregation.

Task:
Classify each writing-evaluation rubric item into exactly
      one criterion role.

Allowed criterion roles:
- foundation
- bonus
- penalty
- activation

Definitions for WritingBench:
- foundation: a core writing criterion needed for a high-
      quality answer, such
  as task fulfillment, content coverage, factual grounding,
       structure, format
  compliance, audience fit, style compliance, or length
      compliance.
- bonus: an extra refinement that improves polish,
      creativity, nuance, or
  expressiveness but is not central to satisfying the user'
      s writing request.
- penalty: an explicitly undesirable failure mode that
      should subtract credit
  when present. Do not use penalty merely because a
      criterion has low score
  bands.
- activation: a context condition whose main role is to
      determine whether
  another criterion is applicable. Use this rarely for
      conditional
  requirements.

Return JSON only with this schema:
{
  "nodes": [
    {"id": "r1", "role": "foundation"}
  ]
}

Rules:
- Use only the provided rubric ids.
- Every rubric id must appear exactly once.
- Most WritingBench checklist items are foundation criteria
      .
- Prefer foundation for style, format, length, and content
      requirements when
  they are explicitly requested by the user.
- Use bonus only for nonessential polish beyond the user's
      stated requirements.
- Use penalty only for criteria that explicitly describe an
       undesirable error or
  violation.
```

WritingBench Edge Annotation Prompt

```
You are annotating directed dependencies among WritingBench
       rubric criteria for
structured reward aggregation.

Task:
For each candidate directed pair, decide whether the parent
       criterion licenses
the child criterion.

Allowed labels:
- none
- weak prerequisite
- strong prerequisite
- activation

Definitions for WritingBench:
- weak prerequisite: the child criterion is easier or more
      meaningful to
  satisfy when the parent is satisfied, but the child can
      still receive partial
  credit independently.
- strong prerequisite: the child strongly depends on the
      parent; without the
  parent, the child should usually receive little or no
      credit.
- activation: the parent determines whether a conditional
      child bonus or penalty
  is applicable.
- none: the two writing criteria are independent dimensions
      .

Writing-specific guidance:
- Content correctness, task fulfillment, and required
      source or material
  coverage can be prerequisites for deeper analysis,
      persuasiveness, or
  domain-specific quality.
- Format, style, and length constraints are often
      independent unless one
  criterion explicitly builds on another.
- Do not add dependencies just because two criteria are
      both important.
- Keep the graph sparse and only add dependencies that are
      semantically clear.

Return JSON only with this schema:
{
  "edges": [
    {"parent": "r1", "child": "r2", "relation": "weak
      prerequisite"}
  ]
}

Rules:
- Use only the provided candidate pairs.
- Return exactly one decision for each candidate pair.
- Use "none" when the relationship is only topical overlap,
       not a dependency.
```

PLawBench Node Annotation Prompt

```
You are annotating PLawBench legal-practice rubrics for
      structured reward
aggregation.

Task:
Classify each legal-evaluation rubric item into exactly one
       criterion role.

Allowed criterion roles:
- foundation
- bonus
- penalty
- activation

Definitions for PLawBench:
- foundation: a core legal-answer criterion such as
      conclusion, fact summary,
  legal reasoning, statutory basis, or legal authorities.
- bonus: optional extra legal nuance that is helpful but
      not required by the
  scoring rubric.
- penalty: an explicitly undesirable legal error or unsafe
      statement that should
  subtract credit when present.
- activation: a context condition whose main role is to
      determine whether
  another criterion is applicable. Use rarely.

Return JSON only with this schema:
{
  "nodes": [
    {"id": "r1", "role": "foundation"}
  ]
}

Rules:
- Use only the provided rubric ids.
- Every rubric id must appear exactly once.
- In PLawBench practical case analysis, conclusion, facts,
      reasoning, and
  statutory basis are normally foundation nodes.
- Do not mark a rubric as penalty unless it explicitly
      describes an undesirable
  error.
```

PLawBench Edge Annotation Prompt

```
You are annotating directed dependencies among PLawBench
      legal-practice rubrics
for structured reward aggregation.

Task:
For each candidate directed pair, decide whether the parent
       criterion licenses
the child criterion.

Allowed labels:
- none
- weak prerequisite
- strong prerequisite
- activation

Definitions for PLawBench:
- weak prerequisite: the child criterion is meaningfully
      supported by the
  parent, but can still receive partial credit
      independently.
- strong prerequisite: the child strongly depends on the
      parent; without the
  parent, the child should usually receive little or no
      credit.
- activation: the parent determines whether a conditional
      child criterion is
  applicable.
- none: the criteria are independent.

Legal-reasoning guidance:
- Fact summary is often a prerequisite for legal reasoning.
- Statutory or legal-authority identification often
      supports legal reasoning.
- Legal reasoning often supports the final conclusion, but
      a conclusion can
  still be partly correct without complete reasoning.
- Do not create reverse dependencies from conclusion to
      facts, reasoning, or
  statutes.
- Keep the graph sparse and avoid redundant dependencies.

Return JSON only with this schema:
{
  "edges": [
    {"parent": "r1", "child": "r2", "relation": "weak
      prerequisite"}
  ]
}

Rules:
- Use only the provided candidate pairs.
- Return exactly one decision for each candidate pair.
- Use "none" when the pair is only part of the same answer
      structure but not a
  dependency.
```

The user prompt supplied to the graph annotator contains the benchmark profile, optional source metadata, the conversation or task prompt, and either the rubric list for role annotation or the candidate directed rubric pairs for dependency annotation. For role annotation, the rubric list contains entries with an id, criterion text, and signed point value. For dependency annotation, each candidate pair contains the parent id, child id, parent role, child role, parent criterion, child criterion, and the dependency labels allowed by the candidate-generation procedure.

J.2 Judge Prompt Templates

Reward judging and final evaluation use the same rubric-level prompt templates. During reinforcement-learning training, the template is instantiated with Qwen3-8B. During final evaluation, the same template is instantiated with Qwen3-32B. The prompt contains the conversation, the candidate assistant response, and the rubric items to be judged. The judge is instructed to return compact machine-readable output only, without explanations, markdown fences, or hidden reasoning.

HealthBench Judge Prompt

```
Your job is to evaluate the last assistant response in a
      conversation against
multiple rubric items.

Input:
- Conversation, including the candidate assistant response.
- A batch of rubric items with signed point values.

Instructions:
- Return exactly one compact machine-readable object and
      nothing else.
- Do not output markdown fences.
- Do not output hidden reasoning or explanations.
- Return one judgment for every required rubric item.
- Do not use rubric ids such as "r1"; use the provided
      numeric rubric keys.
```

```
- The output is invalid if any required rubric item is
      missing.

For each rubric item, report:
- whether the candidate response satisfies the criterion;
- the probability that the criterion is satisfied.

For negative-point rubrics, a satisfied criterion means
      that the undesirable
condition is present.
If the undesirable condition is absent, mark the criterion
      as not satisfied and
assign a probability close to zero.
```

WritingBench and PLawBench Judge Prompt

```
Your job is to evaluate the last assistant response in a
      conversation against
multiple rubric items.

Input:
- Conversation, including the candidate assistant response.
- A batch of rubric items with scoring instructions.

Instructions:
- Return exactly one compact machine-readable object and
      nothing else.
- Do not output markdown fences.
- Do not output hidden reasoning or explanations.
- Return one judgment for every required rubric item.
- Do not use rubric ids such as "r1"; use the provided
      numeric rubric keys.
- The output is invalid if any required rubric item is
      missing.

For each rubric item:
- If the item uses a 1--10 scale, assign a numeric score
      from 1 to 10 according
  to the rubric.
- If the item uses partial points, assign the number of
      awarded points from 0 to
  the maximum points.
- If the item contains explicit scoring subitems, judge
      each subitem
  independently and return either its satisfaction
      probability or its awarded
  points, depending on the scoring mode.

The reward code normalizes the resulting score to the
      interval \([0,1]\) before
aggregation.
```